🚀 Day 9/21: Beginner Python — Functions, Exception Handling & Cleaning Dirty Data

💡 The Data Refinery Daily Motivation (Day 9) "Every expert programmer started by writing simple functions and making beginner mistakes. Don't rush to master advanced tricks until your fundamentals are rock-solid. Let's master the basics today!" ⚡

🏢 Real-World Scenario: The Crashing Dashboard at "InstaDrop" At InstaDrop, store managers submit daily customer ratings (numbers from 1 to 5). A junior analyst wrote: def get_average(ratings): return sum(ratings) / len(ratings)

When a store submitted "N/A", the pipeline crashed with a TypeError! In Data Engineering, data is never clean. We need functions with safety nets.

📖 Part 1: Core Concepts

Functions: Reusable code blocks with input arguments and return values.
Missing Return: If `return` is omitted, Python defaults to returning `None`.
Defensive Coding (`try...except`): Wrap risky data parsing in `try...except` blocks to handle malformed values gracefully.

📱 Part 2: Quick Reference 🔹 TypeError: Mismatched types (e.g., int + str). 🔹 ValueError: Right type, bad value (e.g., int("N/A")). 🔹 ZeroDivisionError: Division by 0 (e.g., empty list average). 🔹 KeyError: Missing dictionary key. Use dict.get('key').

💻 Part 3: Medium-Level Challenge Solution def calculate_safe_average(ratings_list): valid_numbers = [] for item in ratings_list: try: num = float(item) valid_numbers.append(num) except (ValueError, TypeError): continue

```
if not valid_numbers:
    return 0.0
```

```
return round(sum(valid_numbers) / len(valid_numbers), 2)
```

❓ Part 4: 5 MCQs Q1: Unreturned function output -> C) None Q2: Purpose of try…except -> B) Prevent pipeline crashes from bad data Q3: Placeholders in function definition -> B) Parameters Q4: Result of 10 / 0 -> C) ZeroDivisionError Q5: Primary function benefit -> A) Code reusability (DRY principle)

💡 Bonus Technical Q&A Q1: Difference between *args and **kwargs? -> *args passes positional arguments as a tuple; **kwargs passes keyword arguments as a dictionary.

Q2: Purpose of `finally` block? -> Executes code regardless of whether an exception occurred (essential for DB connection/file stream cleanup).

Q3: What is the LEGB scope rule? -> Python looks up variables in order: Local -> Enclosing -> Global -> Built-in.